

Banking System using Object-Oriented Programming in Java

ARITRA TALUKDAR

Roll No: CSB24034

April 29, 2026

Abstract

This project implements a banking system using Object-Oriented Programming (OOP) concepts in Java. The system supports different types of accounts such as Savings and Current accounts, and demonstrates encapsulation, inheritance, polymorphism, constructor chaining, and method overriding.

1 Introduction

A banking system is a real-world application that involves managing accounts, transactions, and user data. This project simulates such a system using Java and focuses on applying OOP principles effectively.

2 Objectives

- To implement a base class **Account**
- To demonstrate **encapsulation** using private fields
- To use **inheritance** for specialized accounts
- To implement **polymorphism** using a list of accounts
- To apply **constructor overloading and chaining**
- To ensure **validation and error handling**

3 OOP Concepts Used

3.1 Encapsulation

Data members are declared private and accessed using getters and setters to ensure data security.

3.2 Inheritance

SavingsAccount and CurrentAccount inherit from the base class Account.

3.3 Polymorphism

A list of Account objects is used to store different types of accounts and invoke methods dynamically.

3.4 Method Overriding

The display() and withdraw() methods are overridden in derived classes.

3.5 Constructor Chaining

The base class uses this() to call another constructor.

4 Implementation

4.1 Code

```
1 import java.util.*;
2
3 // ----- BASE CLASS -----
4 class Account {
5     private String accountNumber;
6     private String ownerName;
7     private double balance;
8
9     public Account() {
10         this("0000", "Unknown", 0.0);
11     }
12
13     public Account(String accountNumber, String ownerName, double
14         balance) {
15         setAccountNumber(accountNumber);
16         setOwnerName(ownerName);
17         setBalance(balance);
18     }
19
20     public String getAccountNumber() {
21         return accountNumber;
22     }
23
24     public void setAccountNumber(String accountNumber) {
25         if (accountNumber == null || accountNumber.isEmpty())
26             throw new IllegalArgumentException("Invalid account
27             number");
28         this.accountNumber = accountNumber;
29     }
30 }
```

```

28
29 public String getOwnerName() {
30     return ownerName;
31 }
32
33 public void setOwnerName(String ownerName) {
34     if (ownerName == null || ownerName.isEmpty())
35         throw new IllegalArgumentException("Invalid owner
36             name");
37     this.ownerName = ownerName;
38 }
39
40 public double getBalance() {
41     return balance;
42 }
43
44 public void setBalance(double balance) {
45     if (balance < 0)
46         throw new IllegalArgumentException("Balance cannot be
47             negative");
48     this.balance = balance;
49 }
50
51 public void deposit(double amount) {
52     if (amount <= 0)
53         throw new IllegalArgumentException("Deposit must be
54             positive");
55     setBalance(getBalance() + amount);
56 }
57
58 public void withdraw(double amount) {
59     if (amount <= 0)
60         throw new IllegalArgumentException("Withdrawal must
61             be positive");
62     if (amount > getBalance())
63         throw new IllegalArgumentException("Insufficient
64             balance");
65     setBalance(getBalance() - amount);
66 }
67
68 public void display() {
69     System.out.println("=====");
70     System.out.println("Account Number: " + getAccountNumber
71         ());
72     System.out.println("Owner Name: " + getOwnerName());
73     System.out.printf("Balance: Rs. %.2f\n",
74         getBalance());
75 }
76 }
77
78 // ----- SAVINGS ACCOUNT -----

```

```

72 class SavingsAccount extends Account {
73     private double interestRate;
74
75     public SavingsAccount(String accNo, String name, double
76         balance, double interestRate) {
77         super(accNo, name, balance);
78         setInterestRate(interestRate);
79     }
80
81     public void setInterestRate(double interestRate) {
82         if (interestRate < 0)
83             throw new IllegalArgumentException("Invalid interest
84                 rate");
85         this.interestRate = interestRate;
86     }
87
88     public double calculateInterest() {
89         return getBalance() * interestRate / 100;
90     }
91
92     @Override
93     public void display() {
94         super.display();
95         System.out.println("Account Type: Savings");
96         System.out.printf("Interest Rate: %.2f%%\n",
97             interestRate);
98         System.out.printf("Interest Earned: Rs. %.2f\n",
99             calculateInterest());
100         System.out.println("=====");
101     }
102 }
103
104 // ----- CURRENT ACCOUNT -----
105 class CurrentAccount extends Account {
106     private double overdraftLimit;
107
108     public CurrentAccount(String accNo, String name, double
109         balance, double overdraftLimit) {
110         super(accNo, name, balance);
111         setOverdraftLimit(overdraftLimit);
112     }
113
114     public void setOverdraftLimit(double overdraftLimit) {
115         if (overdraftLimit < 0)
116             throw new IllegalArgumentException("Invalid overdraft
117                 limit");
118         this.overdraftLimit = overdraftLimit;
119     }
120
121     @Override
122     public void withdraw(double amount) {

```

```

117         if (amount <= 0)
118             throw new IllegalArgumentException("Withdrawal must be positive");
119
120         if (amount > getBalance() + overdraftLimit)
121             throw new IllegalArgumentException("Exceeds overdraft limit");
122
123         setBalance(getBalance() - amount);
124     }
125
126     @Override
127     public void display() {
128         super.display();
129         System.out.println("Account Type: Current");
130         System.out.printf("Overdraft Limit: Rs. %.2f\n",
131             overdraftLimit);
132         System.out.println("=====");
133     }
134 }
135
136 // ----- MAIN CLASS -----
137 public class BankingSystem {
138     public static void main(String[] args) {
139
140         Scanner sc = new Scanner(System.in);
141         List<Account> accounts = new ArrayList<>();
142
143         while (true) {
144             System.out.println("\n==== BANK MENU ====");
145             System.out.println("1. Create Savings Account");
146             System.out.println("2. Create Current Account");
147             System.out.println("3. Deposit");
148             System.out.println("4. Withdraw");
149             System.out.println("5. Display Accounts");
150             System.out.println("6. Exit");
151             System.out.print("Enter choice: ");
152
153             int choice = sc.nextInt();
154             sc.nextLine(); // clear buffer
155
156             try {
157                 switch (choice) {
158                     case 1:
159                         System.out.print("Account Number: ");
160                         String sAcc = sc.nextLine();
161
162                         System.out.print("Owner Name: ");
163                         String sName = sc.nextLine();
164

```

```

165         System.out.print("Initial_Balance:_");
166         double sBal = sc.nextDouble();
167
168         System.out.print("Interest_Rate:_");
169         double rate = sc.nextDouble();
170
171         accounts.add(new SavingsAccount(sAcc,
172             sName, sBal, rate));
173         System.out.println("Savings_Account_
174             Created!");
175         break;
176
177     case 2:
178         System.out.print("Account_Number:_");
179         String cAcc = sc.nextLine();
180
181         System.out.print("Owner_Name:_");
182         String cName = sc.nextLine();
183
184         System.out.print("Initial_Balance:_");
185         double cBal = sc.nextDouble();
186
187         System.out.print("Overdraft_Limit:_");
188         double limit = sc.nextDouble();
189
190         accounts.add(new CurrentAccount(cAcc,
191             cName, cBal, limit));
192         System.out.println("Current_Account_
193             Created!");
194         break;
195
196     case 3:
197         Account depAcc = findAccount(accounts, sc
198             );
199         if (depAcc != null) {
200             System.out.print("Amount_to_deposit:_
201                 ");
202             double amt = sc.nextDouble();
203             depAcc.deposit(amt);
204             System.out.println("Deposit_
205                 successful!");
206         }
207         break;
208
209     case 4:
210         Account wAcc = findAccount(accounts, sc);
211         if (wAcc != null) {
212             System.out.print("Amount_to_withdraw:
213                 _");
214             double amt = sc.nextDouble();
215             wAcc.withdraw(amt);

```

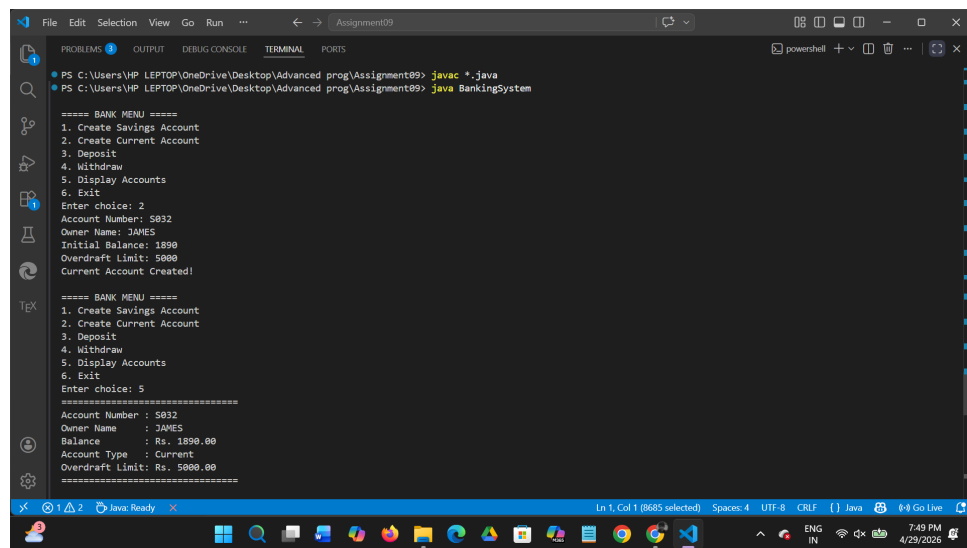
```

208         System.out.println("Withdrawal_
                successful!");
209     }
210     break;
211
212     case 5:
213         if (accounts.isEmpty()) {
214             System.out.println("No_accounts_found
                .");
215         } else {
216             for (Account acc : accounts) {
217                 acc.display();
218             }
219         }
220         break;
221
222     case 6:
223         System.out.println("Thank_you!");
224         return;
225
226     default:
227         System.out.println("Invalid_choice!");
228     }
229
230     } catch (Exception e) {
231         System.out.println("Error:_ " + e.getMessage());
232         sc.nextLine(); // prevent crash
233     }
234 }
235
236
237 // Helper method
238 public static Account findAccount(List<Account> accounts,
    Scanner sc) {
239     System.out.print("Enter_Account_Number:_");
240     String accNo = sc.nextLine();
241
242     for (Account acc : accounts) {
243         if (acc.getAccountNumber().equals(accNo)) {
244             return acc;
245         }
246     }
247
248     System.out.println("Account_not_found!");
249     return null;
250 }
251 }

```

5 Output

5.1 Sample Output Screenshots



The screenshot shows a Windows terminal window with the title bar 'Assignment09'. The terminal displays the output of a Java program. At the top, two PowerShell commands are shown: 'javac *.java' and 'java BankingSystem'. The program output starts with '==== BANK MENU ====', followed by a numbered list of options: 1. Create Savings Account, 2. Create Current Account, 3. Deposit, 4. Withdraw, 5. Display Accounts, 6. Exit. The user enters '2'. The program then prompts for 'Account Number: 5832', 'Owner Name: JAMES', 'Initial Balance: 1899', and 'Overdraft Limit: 5000'. It then outputs 'Current Account Created!'. The menu is displayed again, and the user enters '5'. The program then displays account details: 'Account Number : 5832', 'Owner Name : JAMES', 'Balance : Rs. 1899.00', 'Account Type : Current', and 'Overdraft Limit: Rs. 5000.00'. The terminal window has a status bar at the bottom showing 'Ln 1, Col 1 (6685 selected) Spaces: 4 UTF-8 CRLF Java 8:09 Go Live' and a Windows taskbar at the very bottom with the time '7:49 PM' and date '4/29/2026'.

```
PS C:\Users\HP_LEPTOP\OneDrive\Desktop\Advanced prog\Assignment09> javac *.java
PS C:\Users\HP_LEPTOP\OneDrive\Desktop\Advanced prog\Assignment09> java BankingSystem

==== BANK MENU ====
1. Create Savings Account
2. Create Current Account
3. Deposit
4. Withdraw
5. Display Accounts
6. Exit
Enter choice: 2
Account Number: 5832
Owner Name: JAMES
Initial Balance: 1899
Overdraft Limit: 5000
Current Account Created!

==== BANK MENU ====
1. Create Savings Account
2. Create Current Account
3. Deposit
4. Withdraw
5. Display Accounts
6. Exit
Enter choice: 5
=====
Account Number : 5832
Owner Name : JAMES
Balance : Rs. 1899.00
Account Type : Current
Overdraft Limit: Rs. 5000.00
=====
```

Figure 1: Program Output Screenshot

6 Results

The system successfully demonstrates:

- Account creation
- Deposit and withdrawal operations
- Interest calculation
- Overdraft handling
- Error handling for invalid inputs

7 Conclusion

The project effectively demonstrates core OOP principles in Java. It provides a modular and extensible design that can be further enhanced with database integration or GUI.

8 Future Enhancements

- Add database support (MySQL/JDBC)
- Implement GUI using JavaFX
- Add authentication system
- Maintain transaction history